Design and Planning

First error came up.

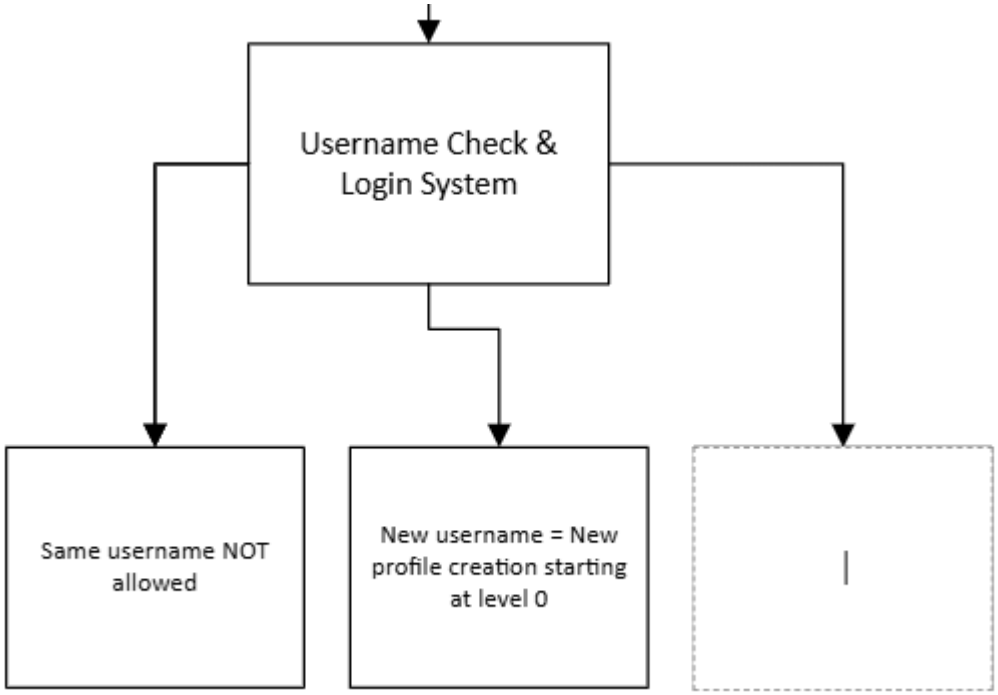


Figure 1 – Flawed Username System Logic Error Spotted.

I didn't realise at first but whilst creating the structure diagram, I realised that how can you not have duplicate usernames but also load if it exists. At this point it became inevitable to opt in for a username and password system that'd prevent this, which I was trying to cut corners with at first by only creating a username.

Fix:-

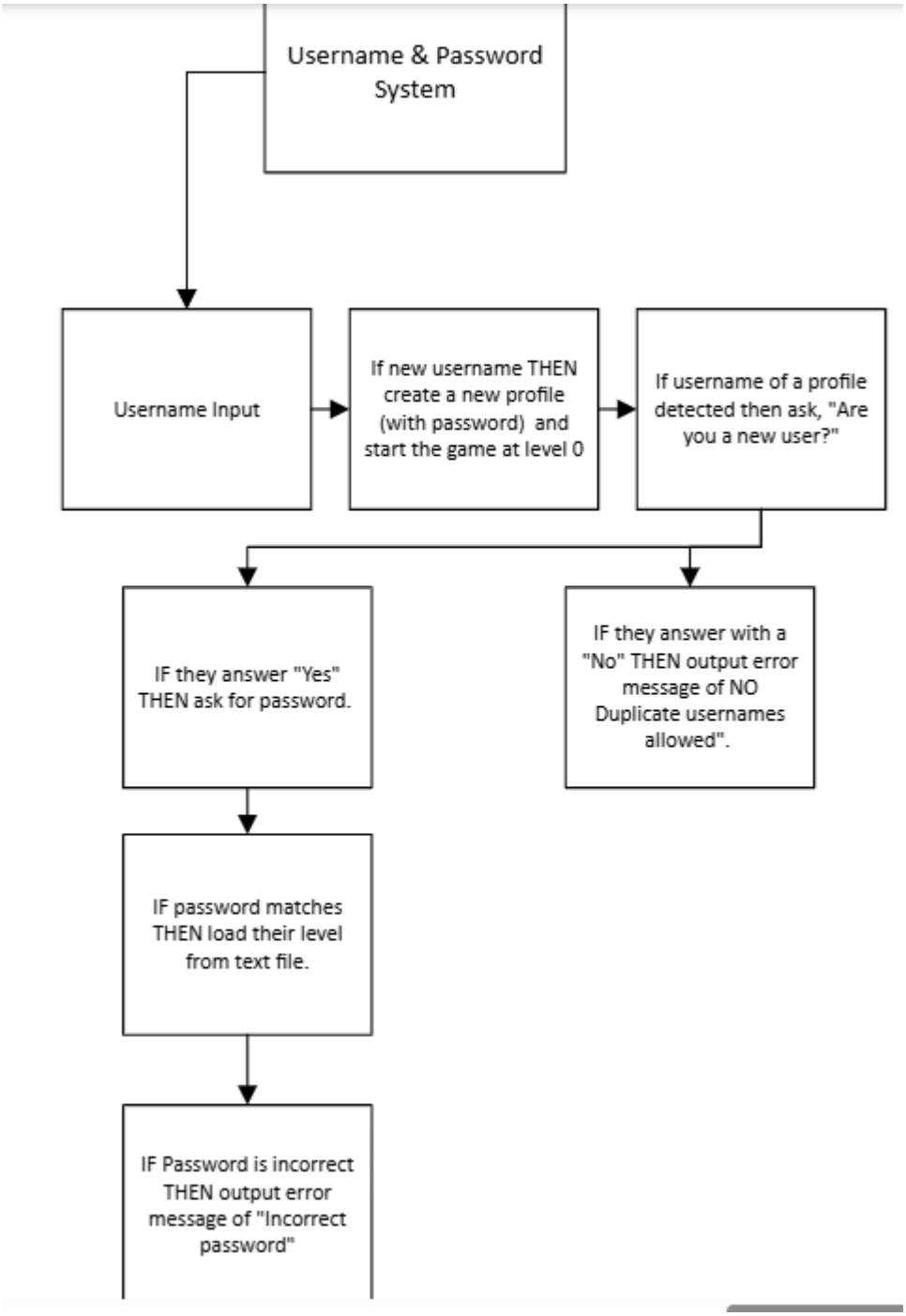


Figure 1.2 – Fix for not having the same usernames.

Gameplay logic improvement.

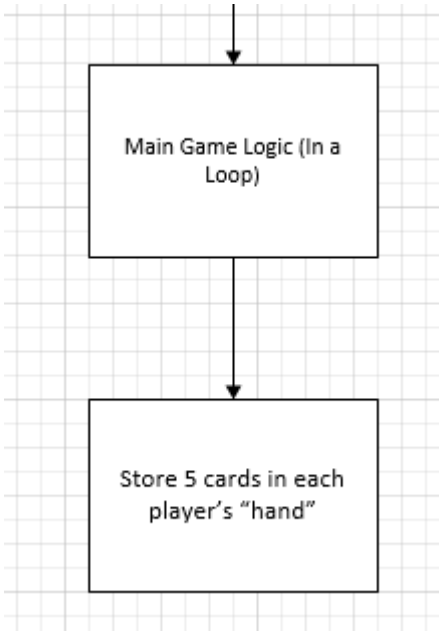


Figure 2 – Less Efficient Game Logic

As I came across the third paragraph in the assignment brief I realised I missed out more components of the "card system". I thought that everything related to the card creation was done but I found out about two more points in the third paragraph: - The deck of card which has 10 cards should be shuffled randomly and split evenly amongst the player and the computer.

I thought I'd just add it into the gameplay section which I can but it wouldn't be as logical and as readable compared to if I added these requirements within the "card system" section / function of the game as that'd make more sense.

Fix : -

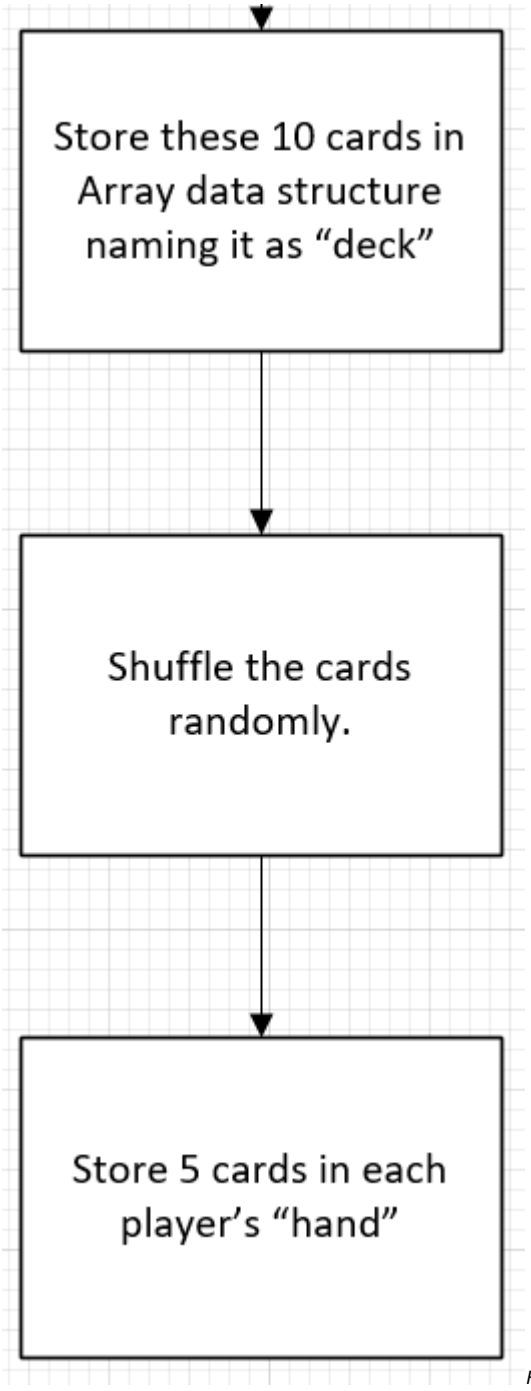


Figure 2.2 – Card System expanded.

Problem 3

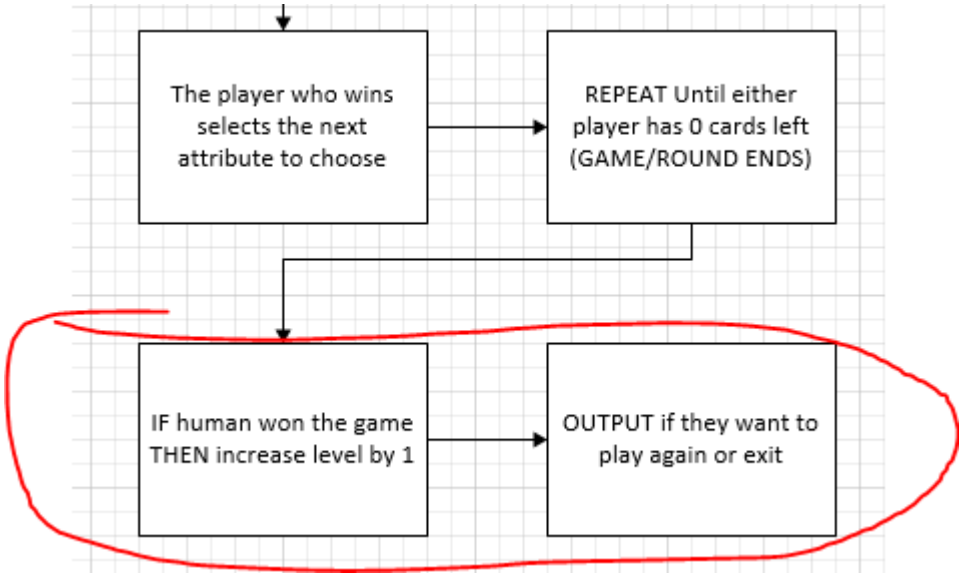


Figure 3 – End Game Confusion

As I was working through the design of the main game loop logic, I wanted to end it here so that there are less modules overall and truthfully there isn't much more to the game after this.

However, upon some contemplation I thought that this is getting too messy, and this doesn't even seem right because the loop ends when the cards reach 0 so adding these conditions to it would prove that the structure diagram isn't an accurate representation of the actual code.

Hence, I have decided to create a separate module for "End game" which will have these last elements of the game, with more thought given to the user experience side of things.

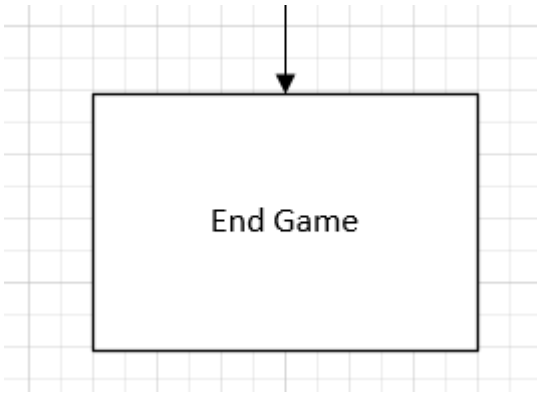


Figure 3.2 – A separate module created for End Game

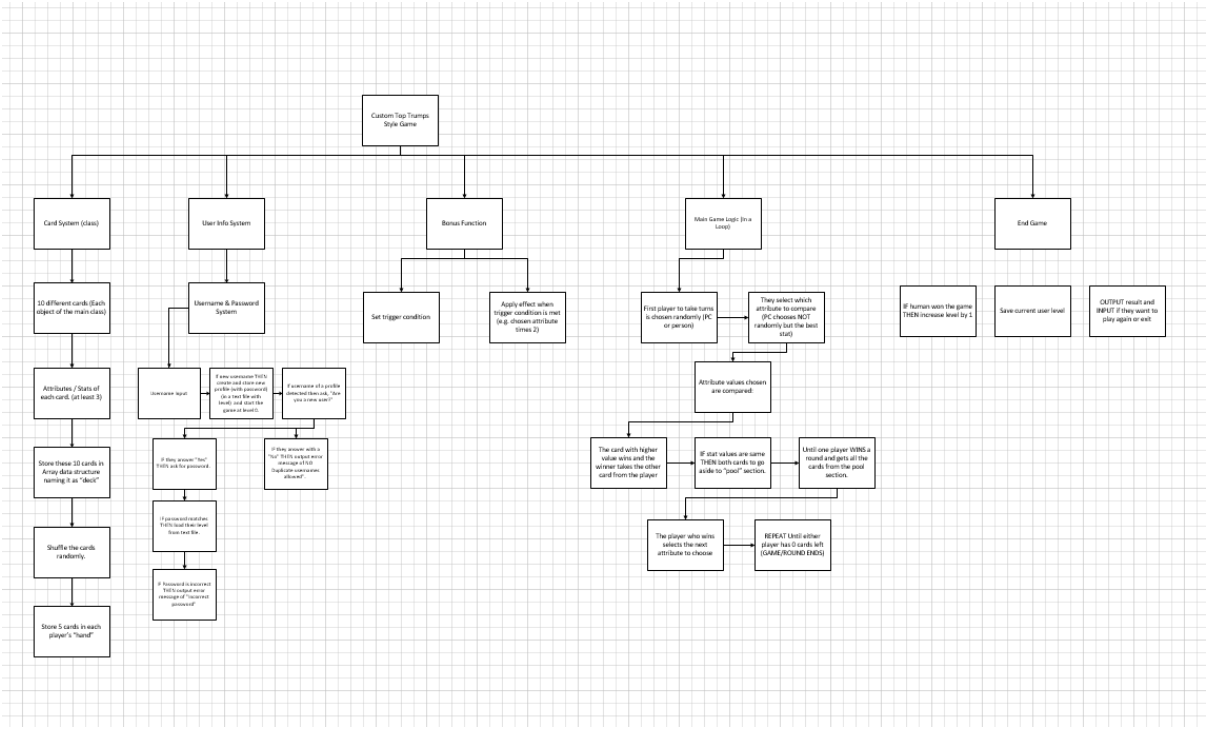


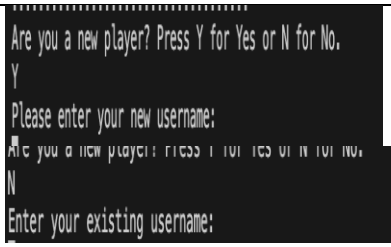
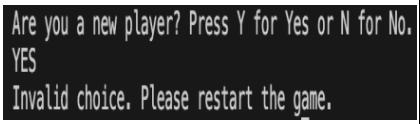
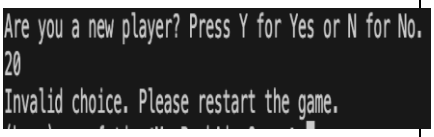
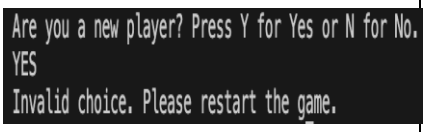
Figure 4.0 – An Eagle Eye view of the stepwise refinement design (COMPLETE).

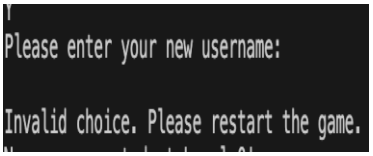
Pseudocode

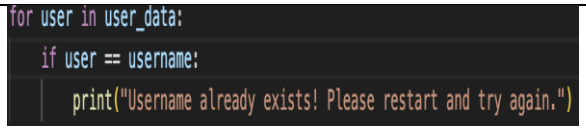
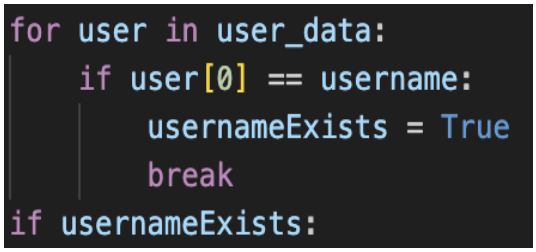
Pseudocode for Dragon Ball Z.txt

Testing & Development

Test Data	Test Description	Test Data	Expected Result	Actual Result

1	Input validation check
	
	`if userType == "Y":`
	`  print("Please enter your new username:")`
	`elif userType == "N":`
	`  print("Enter your existing username:")`
		Valid = “Y” “N”
		
		Invalid. = YES 20 “empty”
			For the game to give an according response to any non-sensical input.	
				PASSED - Works as expected. Upon pressing Y the game then proceeds to ask for their username or if N is pressed then it asks for their **existing** username.
				
				 
				
				 
				
				 
				
				Otherwise it says “invalid choice”

2	`print("New user created at Level 0!")`
	`file = open(filename, "a")`
	`file.write(username + "," + str(level) + "\n")`
	`file.close()`
	
	New username creation check.
		Valid = “**Hotboi69**”
		
		Invalid = Blank
			For the game to create a new user with the given username (given that it’s not been used before) and to save it in the user database file.	
				
				 
				
				PASSED - So far so good, the game is not only saving the data but also lets the user know too that their progress will now be saved.

2.2	 Test to see if the username is actually saved after I try to log in from it again.	Valid = Same username as before Invalid = New username	For the game to respond with an error message saying that the username already exists.	 FAILED - Unfortunately, for some reason this isn't working and it keep resulting back in "new user created".
2.3	Test to see if the username not being saved error caused in 2.2 is fixed or not.  This part was the problematic part. This was a typo, as I meant to put the [0] next to the user not the username. However this caused a tremendous amount of stress.  Also added another variable called usernameExists through which I based off another set of nested loop statements and got this to work.	N/A	For the game to respond with an error message saying that the username already exists.	 As expected this time the error was fixed. PASSED

3	Testing game logic; if a username is inputted that isn't recognized the game should deny further entry into the game.
	
	`if not found:`
	`  print("Username not found! Please restart the game.")`
	`print(f"Welcome back {username}! Your current level is {level}.")`
		"bOB"
		"sexyboi69"
		"
			For the game to stop at this point (end) so that the user can run it again to rectify their response.	`Username not found! Please restart the game.`
				`Welcome back shamkalalal! Your current level is 0.`
				
				This is incorrect game logic, the game should just end here or restart on its own, but it should certainly not assign them a new level after an incorrect username check.

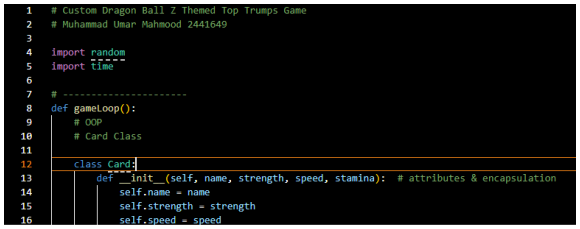
3.2	Test to see if existing username != found
	
	`found = False`
	`for user in user_data:`
	`  if user[0] == username:`
	`  level = int(user[1])`
	`  found = True`
	`if not found:`
	`  print("Username not found! Please restart the game.")`
	`elif found == True:`
	`  print(f"Welcome back {username}! Your current level is {level}.")`
	`elif username == "":`
	`  print ("Invalid choice. Please restart the game.")`
	`else:`
	`  print("Invalid choice. Please restart the game.")`
		Valid = "ofjjf" "jfni12" "12"
		
		Invalid = Blank
			For the game to detect that it's not in the database (the player is LYING!) and to tell them to restart the game, whilst also catching any invalid inputs too	FAILED – Although initially it did spot any username attempts that were not in the database and reflected this within the answer it gave out but it did not catch any blank statements.
				
				`N`
				`Enter your existing username:`
				`JDJ`
				`Username not found! Please restart the game.`
				
				`N`
				`Enter your existing username:`
				
				`Welcome back ! Your current level`

3.3	Test to see if existing username != found	Valid = "omr" "omr43"	For the game to detect that it's not in the	`Enter your existing username:`
				`omr`
				`Username not found! Please restart the game.`

	``
	else:
	    found = False
	    for user in user_data:
	        if user[0] == username:
	            level = int(user[1])
	            found = True
	    if found:
	        print(f"Welcome back {username}! Your current level is {level}.")
	    else:
	        print("Username not found! Please restart the game.")
	``
	
	Whilst also checking for invalid inputs like leaving it blank.
	
	``
	username = input()
	if username == "":
	    print("Invalid choice. Please restart the game.")
	``
		Invalid = blank	database (the player is LYING!) and to tell them to restart the game whilst also catching any invalid inputs too	As expected the game does just exactly that.
				
				``
				Are you a new player? Press Y for Yes or N for No.
				N
				Enter your existing username:
				
				Invalid choice. Please restart the game.
				``
				
				PASSED
				
				HOWEVER, here there is room for improvement – there could be a password feature here too that if the system does detect that the username does exist in its database then it should ask for a password.

4	```
	if found:
	    print(f"Wel
	    gameLoop()
	``
	
	Testing main game loop. To see if after a successful login or a successful user creation, does the program go on towards the main game loop.
		N/A	For the code to go straight on towards the main game loop after its been called at the login system.	```
				NameError: name 'gameLoop' is not defined
				``
				
				FAILED – The code does not recognise gameLoop() function as I had initially thought of including it later down in the program when the round starts etc. However this is not working as the python doesn't recognise gameLoop() at that stage yet.
				
				Therefore, the code only works till getting their usernames and such and does not work beyond that.

4.2	Game Loop Tests. ``` 1 # Custom Dragon Ball Z Themed Top Trumps Game 2 # Muhammad Umar Mahmood 2441649 3 4 import random 5 import time 6 7 # ---------------------- 8 # Previous User's Login Data (File Handling) 9 filename = "userData.txt" 10 user_data = [] # Empty array, here the user data shall be saved. 11 #This way I can also use array indexing to access different attributes / features of the card(s 12 13 file = open(filename, "r") 14 for line in file: 15 line = line.strip() 16 usernameNLvl = line.split(",") 17 user_data.append(usernameNLvl) 18 file.close() 19 20 # ---------------------- 21 #Game Instructions (User experience focused) 22 print("**********************************") 23 print("WELCOME TO DRAGON BALL Z TOP TRUMPS! THE ULTIMATE GAME!") ```	N/A	As I have now changed the order of almost the whole code up until now so that the function of gameLoop() can be	The code still does not work past that as this has led to newer multiple issues regarding encapsulation (private).

			defined first and when its called later in the Login system it works as intended.	

Final Conclusion

In the final conclusion I'd say that this project was a good practice to enhance my coding skills, especially that of OOP which I had to learn quite a lot for this.

It's a shame that despite all this I ran into issues that I couldn't fix in the amount of time I though it'd take me to complete this project, hence why I am having to now submit the assignment even though it is incomplete.

Some of the things that could be used for the improvement of this game that I could think of was to add some sort of a security element to this game and hence add a password like structure for returning users.

Other than that more power ups or "booster" cards which could turn the tide of the round for a player.

References

https://www.w3schools.com/python/python_file_handling.asp

https://www.geeksforgeeks.org/how-to-split-a-file-into-a-list-in-python/

https://isaaccomputerscience.org/topics/object_oriented_programming

https://www.youtube.com/playlist?list=PL-osiE80TeTsqhIuOqKhwlXsIBIdSeYtc

https://www.datacamp.com/tutorial/python-remove-item-from-list